E-commerce Application

Repo URL : https://github.com/wikkics/ecommerce-app.git

Tools Installed

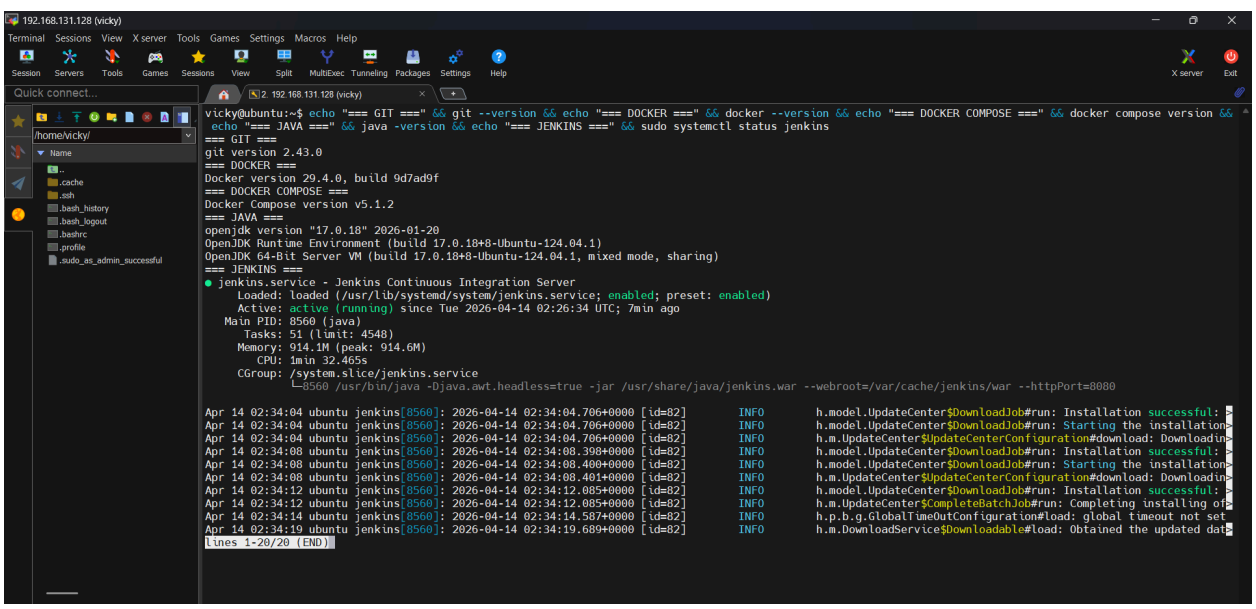
Git — I installed Git to clone the project repository from GitHub and to handle all version control operations using the command line throughout the project.

Docker — I installed Docker to containerize the React application. Docker allows the application to run consistently in any environment by packaging it along with all its dependencies into a container.

Docker Compose — I installed Docker Compose alongside Docker to manage and run the container using a simple configuration file instead of long Docker commands.

Java JDK 17 — I installed Java because Jenkins requires it to run. OpenJDK 17 is the recommended version for the latest Jenkins releases.

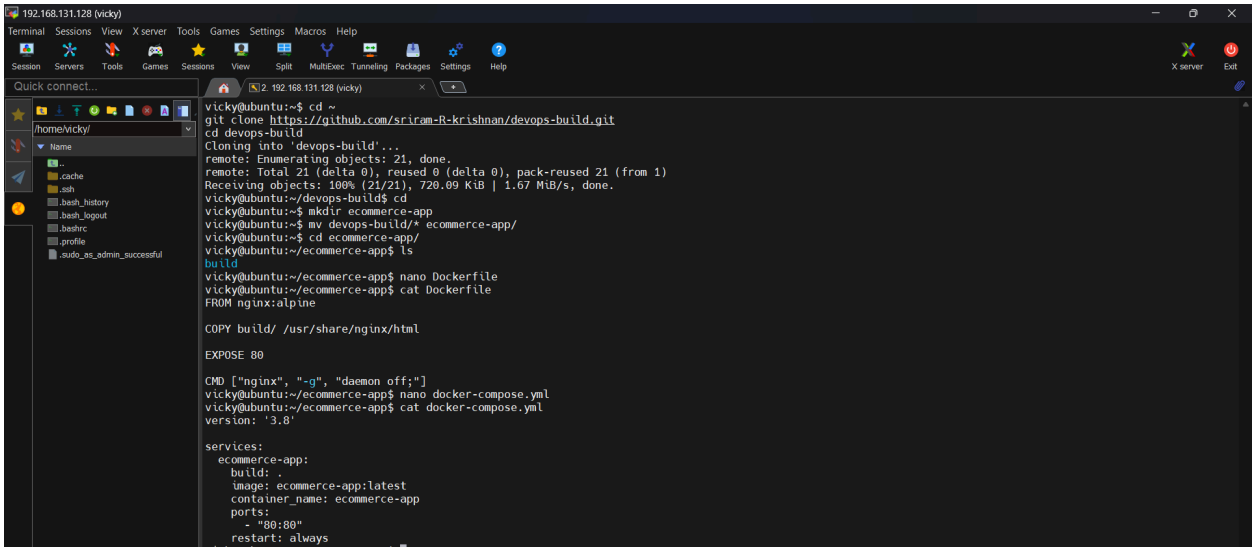
Jenkins — I installed Jenkins to set up the CI/CD pipeline. Jenkins will automatically build and push Docker images whenever code is pushed to GitHub, removing the need for manual intervention.

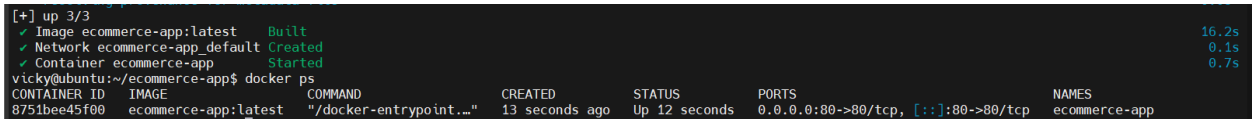


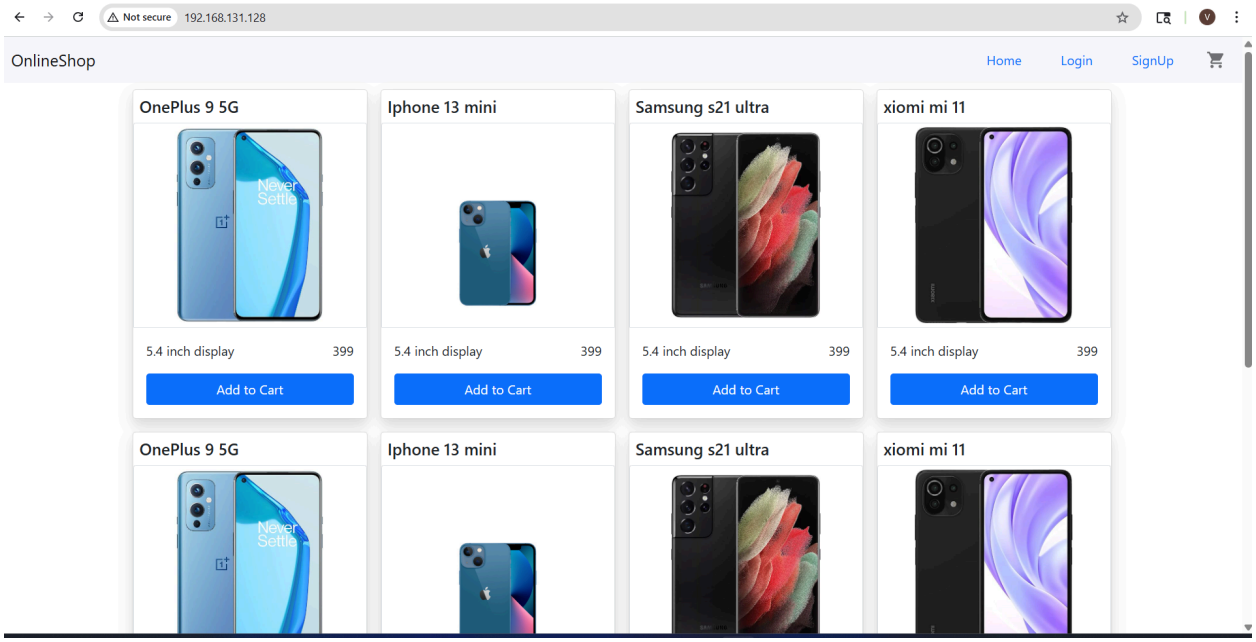
I cloned the repository from GitHub and renamed the folder to `ecommerce-app`. After examining the project structure, I found that the repository already contained a pre-built React application inside the `build/` folder, so Node.js installation was not required.

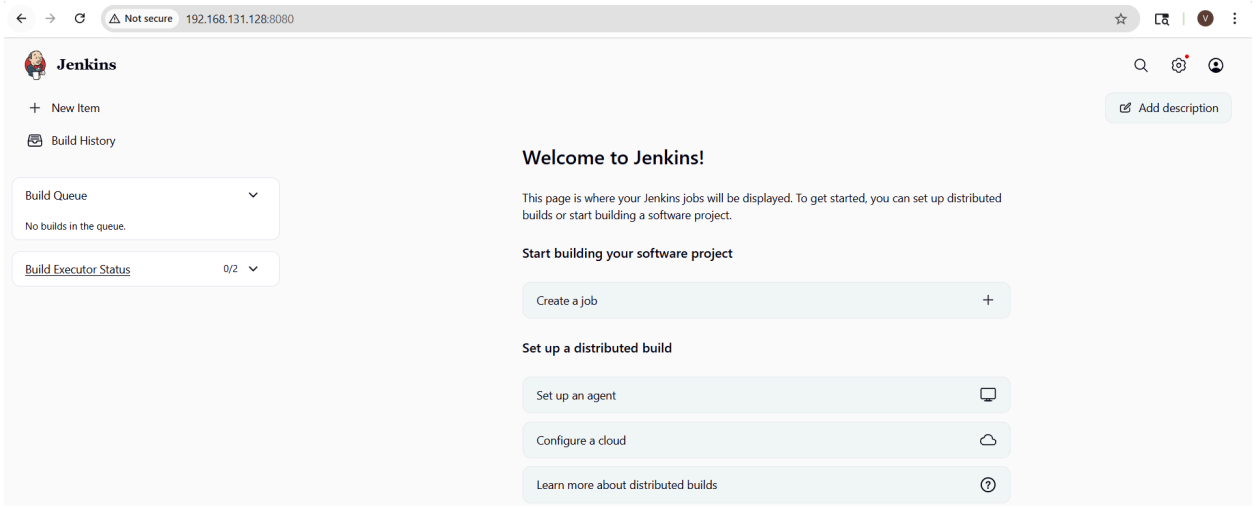
I created a `Dockerfile` using `nginx:alpine` as the base image to directly serve the static files from the `build/` folder on port 80. I also created a `docker-compose.yml` file to manage the container with the name `ecommerce-app`, mapped to port 80 with `restart: always` to ensure it stays running.

Finally, I built and started the container using `docker compose up --build -d` and verified it was running using `docker ps`. The application was successfully accessible at `http://192.168.131.128/` in the browser. Jenkins was also confirmed running separately at `http://192.168.131.128:8080` without any port conflict.



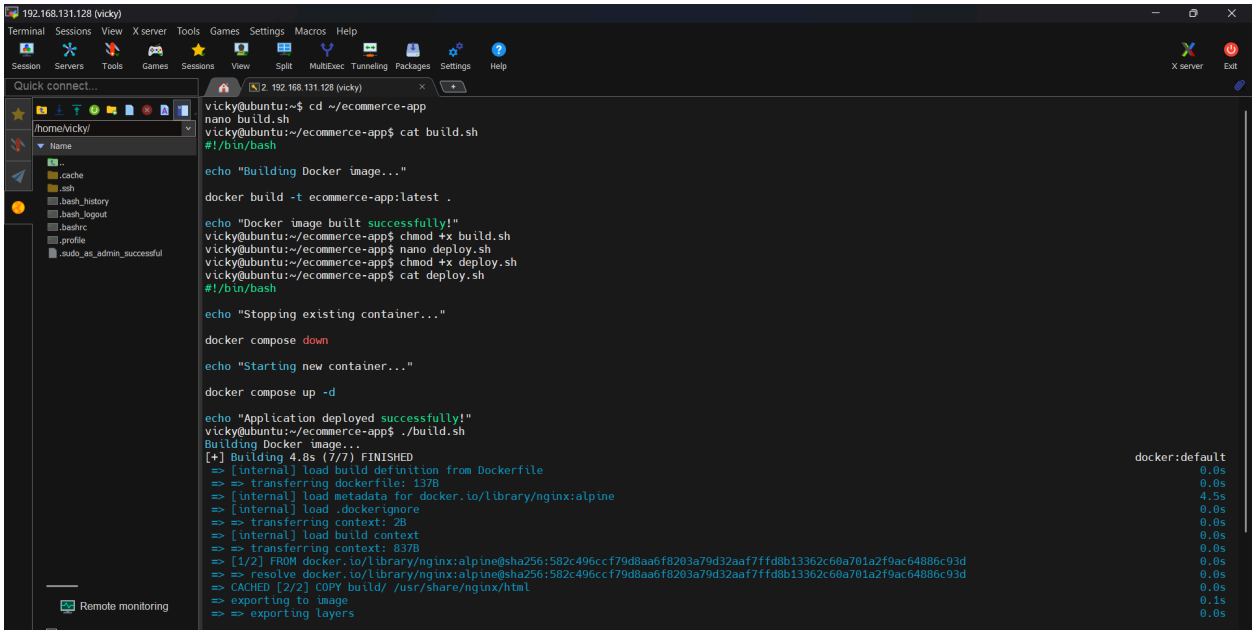


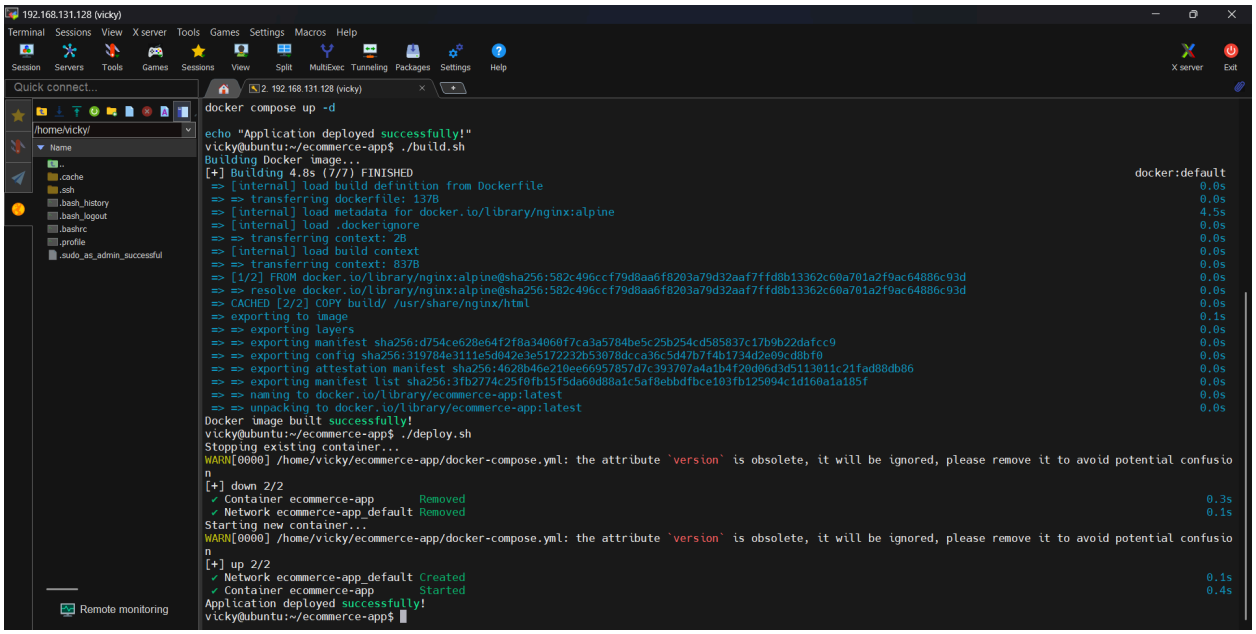




I created two bash scripts to automate the Docker build and deployment process. The `build.sh` script builds the Docker image and the `deploy.sh` script stops the existing container and starts a new one using docker-compose.

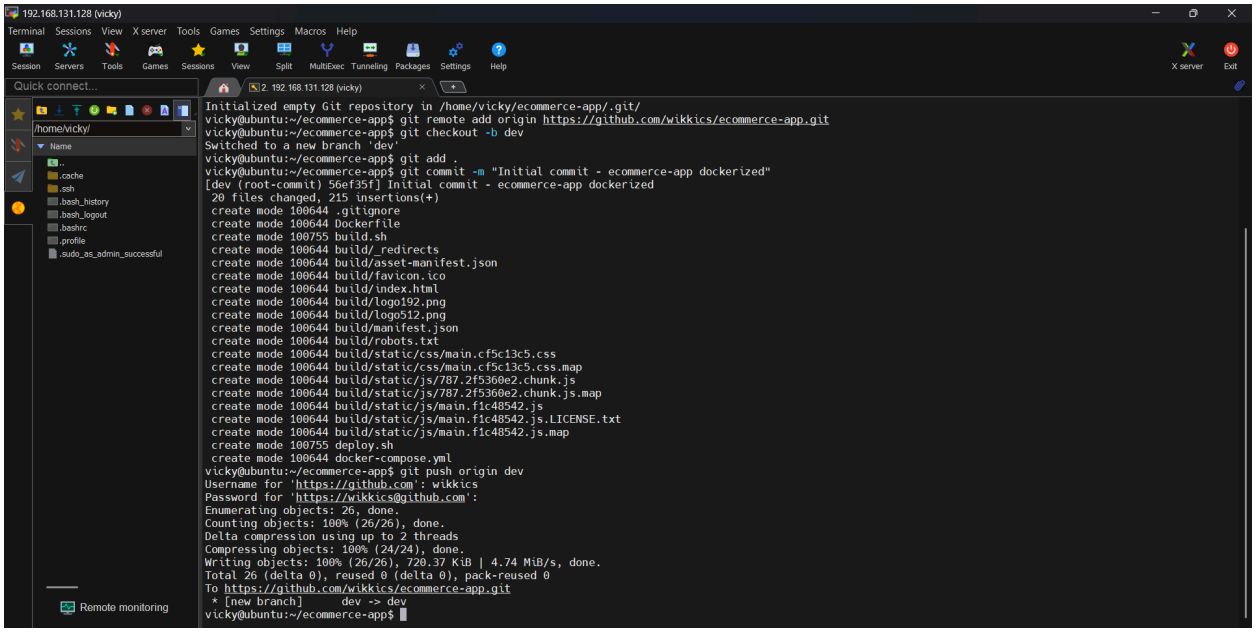
I created both scripts inside the `ecommerce-app` folder, made them executable using `chmod +x` and tested them by running `./build.sh` and `./deploy.sh` one by one. Both scripts ran successfully and the application was confirmed running at `http://192.168.131.128/` after deployment.

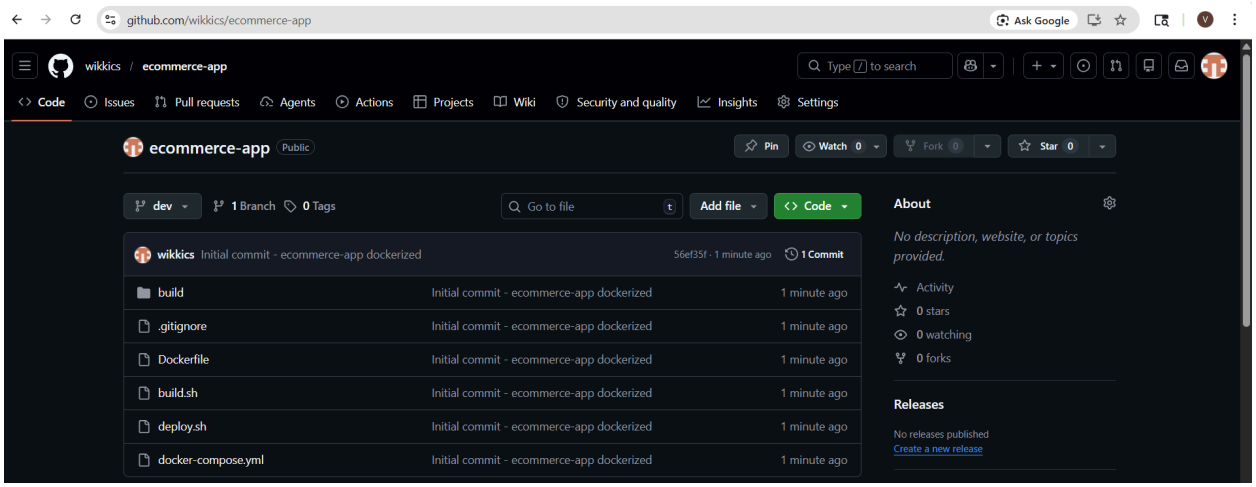




I initialized a new Git repository inside the `ecommerce-app` folder and connected it to my own GitHub account `wikkics`. I created a new public repository named `ecommerce-app` on GitHub and pushed the code to the `dev` branch using the terminal with my Personal Access Token for authentication.

The pushed code included the `Dockerfile`, `docker-compose.yml`, `build.sh`, `deploy.sh` and the pre-built React application files. All version control operations were performed using Git CLI only without any GUI tools





I created two repositories on Docker Hub under my account `wikkics`. The first repository `ecommerce-app-dev` was created as **public** for development images and the second repository `ecommerce-app-prod` was created as **private** for production images.

I logged into Docker Hub from the terminal using `docker login`, tagged the locally built image as `wikkics/ecommerce-app-dev:latest` and pushed it to the dev repository successfully.

hub.docker.com/repositories/wikkics

New Insights on the state of AI agents from 800+ builders and leaders. Download your copy →

hub | Explore | My Hub | Search Docker Hub | Ctrl K

wikkics
Docker Personal

- Repositories
- Hardened Images
- Collaborations
- Settings
  - Default privacy
  - Notifications
- Billing
- Usage
  - Pulls
  - Storage

Upgrade plan

Repositories

All repositories within the wikkics namespace.

Search by repository name | All content | Create a repository

Name	Last Pushed	Contains	Visibility	Scout
wikkics/ecommerce-app-prod	1 minute ago		Private	Inactive
wikkics/ecommerce-app-dev	1 minute ago		Public	Inactive
wikkics/trend-app	3 days ago	IMAGE	Public	Inactive
wikkics/barista	29 days ago	IMAGE	Public	Inactive
wikkics/parallax	29 days ago	IMAGE	Public	Inactive

1–5 of 5

192.168.131.128 (vicky)

Terminal Sessions View X server Tools Games Settings Macros Help

Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help X server Exit

Quick connect...

2. 192.168.131.128 (vicky)

/home/vicky/

Name
..
.cache
.ssh
.bash_history
.bash_logout
.bashrc
.profile
.sudo_as_admin_successful

```
vicky@ubuntu:~/ecommerce-app$ docker tag ecommerce-app:latest wikkics/ecommerce-app-dev:latest
vicky@ubuntu:~/ecommerce-app$ docker push wikkics/ecommerce-app-dev:latest
The push refers to repository [docker.io/wikkics/ecommerce-app-dev]
ff9f59a6a62e: Pushed
a71873b303e8: Pushed
8b9447c3b2e5: Pushed
8ea687ccfe0b: Pushed
589002ba0eae: Pushed
f03becc8ac15: Pushed
34dfdd2ef1f9: Pushed
c8a2fa3a88d2: Pushed
1165b869c51a: Pushed
15e759724ff6: Pushed
latest: digest: sha256:3fb2774c25f0fb15f5da60d88a1c5af8ebbdfbce103fb125094c1d160a1a185f size: 856
vicky@ubuntu:~/ecommerce-app$ docker images
```

Info → U In Use

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ecommerce-app:latest	3fb2774c25f0	96MB	26.7MB	U
wikkics/ecommerce-app-dev:latest	3fb2774c25f0	96MB	26.7MB	U

```
vicky@ubuntu:~/ecommerce-app$
```

hub.docker.com/repository/docker/wikkics/ecommerce-app-dev/general

New Insights on the state of AI agents from 800+ builders and leaders. Download your copy →

hub | Explore | My Hub | Search Docker Hub | Ctrl K

wikkics
Docker Personal

- Repositories
- Hardened Images
- Collaborations
- Settings
  - Default privacy
  - Notifications
- Billing
- Usage
  - Pulls
  - Storage

Upgrade plan

Repositories / ecommerce-app-dev / General

wikkics/ecommerce-app-dev

Last pushed 1 minute ago · Repository size: 25.5 MB · 0 · 0

Add a description

Add a category

General | Tags | Image Management | Collaborators | Webhooks | Settings

Tags

DOCKER SCOUT INACTIVE
Activate

This repository contains 0 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	1 minute

See all

Docker commands

Public view

To push a new tag to this repository:

```
docker push wikkics/ecommerce-app-dev:tagname
```

buildcloud

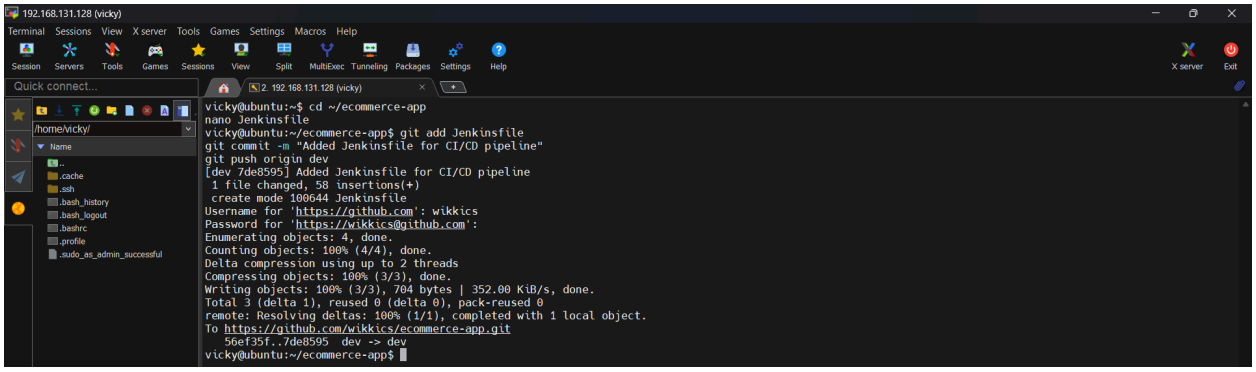
Build with
Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without

I created a `Jenkinsfile` inside the `ecommerce-app` folder which defines the complete CI/CD pipeline. The pipeline has four stages — cloning the repository, building the Docker image, pushing to the dev Docker Hub repo when code is pushed to the `dev` branch, and pushing to the prod Docker Hub repo when code is merged to the `master` branch. After creating the Jenkinsfile, I pushed it to the `dev` branch on GitHub using `git add`, `git commit` and `git push` commands.



I created a new pipeline job in Jenkins named `ecommerce-app-pipeline` and configured it to use the `Jenkinsfile` from the GitHub repository `wikkics/ecommerce-app`. I enabled the GitHub hook trigger for GITScm polling option so that Jenkins automatically triggers a build whenever code is pushed to GitHub.

I then added a webhook in the GitHub repository settings with the Jenkins URL `http://192.168.131.128:8080/github-webhook/` so that GitHub notifies Jenkins on every code push. This completes the auto-trigger setup between GitHub and Jenkins.

I created a Jenkins pipeline job named `ecommerce-app-pipeline` and configured it to watch both `*/dev` and `*/master` branches using Poll SCM with a schedule of `* * * * *` which checks GitHub every minute for new changes. Since the machine uses a local IP `192.168.131.128`, GitHub webhooks could not reach Jenkins directly, so Poll SCM was used as an alternative which works perfectly with local network setups.

After successful configuration, I tested the pipeline by pushing code to both branches. Pushing to `dev` branch automatically triggered Jenkins which built the image and pushed it to `ecommerce-app-dev` on Docker Hub. Pushing to `master` branch automatically triggered Jenkins which built the image and pushed it to `ecommerce-app-prod` on Docker Hub. Both pipelines completed successfully with green status.

Not secure 192.168.131.128:8080/job/ecommerce-app-pipeline/configure

Jenkins / ecommerce-app-pipeline / Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/wikkics/ecommerce-app.git

Credentials ?

wikkics/******

+ Add

Advanced

+ Add Repository

Branches to build ?

Save Apply

github.com/wikkics/ecommerce-app/settings/hooks/new

wikkics / ecommerce-app

Type / to search

Code Issues Pull requests Agents Actions Projects Wiki Security and quality Insights Settings

- General

Access

- Collaborators
- Moderation options

Code and automation

- Branches
- Tags
- Rules
- Actions
- Models Preview
- Webhooks
- Copilot
- Environments
- Codespaces
- Pages

Security and quality

- Advanced Security

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, `x-www-form-urlencoded`, *etc*). More information can be found in our developer documentation.

Payload URL *

http://192.168.131.128:8080/github-webhook/

Content type *

application/json

Secret

SSL verification

By default, we verify SSL certificates when delivering payloads.

Enable SSL verification Disable (not recommended)

Which events would you like to trigger this webhook?

Just the `push` event.

Send me **everything**.

Not secure 192.168.131.128:8080/job/ecommerce-app-pipeline/5/

Jenkins / ecommerce-app-pipeline / #5

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#5'
- Polling Log
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces
- Previous Build

#5 (14-Apr-2026, 3:53:07 am)

Add description Keep this build forever

Started by an SCM change

Started 59 sec ago
Took 8.6 sec

This run spent:

- 5.6 sec waiting;
- 8.6 sec build duration;
- 14 sec total from scheduled to completion.

git

Revision: 13fa55ac429287d13b5632d75f2dc7228ca53717
Repository: https://github.com/wikkics/ecommerce-app.git

- refs/remotes/origin/dev

Test auto trigger 07a795e
vigneshcsekaran at 3:43 am 14/04/26

Test auto trigger 758e8e8
vigneshcsekaran at 3:46 am 14/04/26

Test poll SCM trigger 13fa55a
vigneshcsekaran at 3:52 am 14/04/26

Not secure 192.168.131.128:8080/job/ecommerce-app-pipeline/7/

Jenkins / ecommerce-app-pipeline / #7

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#7'
- Polling Log
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces
- Previous Build

#7 (14-Apr-2026, 4:06:07 am)

Add description | Keep this build forever

Started by an SCM change

Started 1 min 3 sec ago
Took 22 sec

This run spent:

- 5.7 sec waiting;
- 22 sec build duration;
- 27 sec total from scheduled to completion.

git

Revision: 5a38c22ab9409c9bb3adc279056a52d1be5e2813
Repository: https://github.com/wikkics/ecommerce-app.git

- refs/remotes/origin/master

Test master auto trigger 5a38c22
vigneshcsekaran at 4:04 am 14/04/26

REST API Jenkins 2.541.3

192.168.131.128:8080/job/ecommerce-app-pipeline/7/changes

hub.docker.com/repository/docker/wikkics/ecommerce-app-prod/general

New Insights on the state of AI agents from 800+ builders and leaders. Download your copy →

hub | Explore | My Hub | Search Docker Hub Ctrl K

wikkics
Docker Personal

- Repositories
- Hardened Images
- Collaborations
- Settings
  - Default privacy
  - Notifications
- Billing
- Usage
  - Pulls
  - Storage

Upgrade plan

Repositories / ecommerce-app-prod / General

wikkics/ecommerce-app-prod

Last pushed 1 minute ago • Repository size: 25.5 MB • 0 • 0

Add a description

Add a category

General | Tags | Image Management | Collaborators | Webhooks | Settings

Tags

DOCKER SCOUT INACTIVE
Activate

This repository contains 0 tag(s).

Tag	OS	Type	Pulled	Pushed
latest		Image	less than 1 day	1 minute

See all

Docker commands

To push a new tag to this repository:

```
docker push wikkics/ecommerce-app-prod:tagname
```

buildcloud

Build with
Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

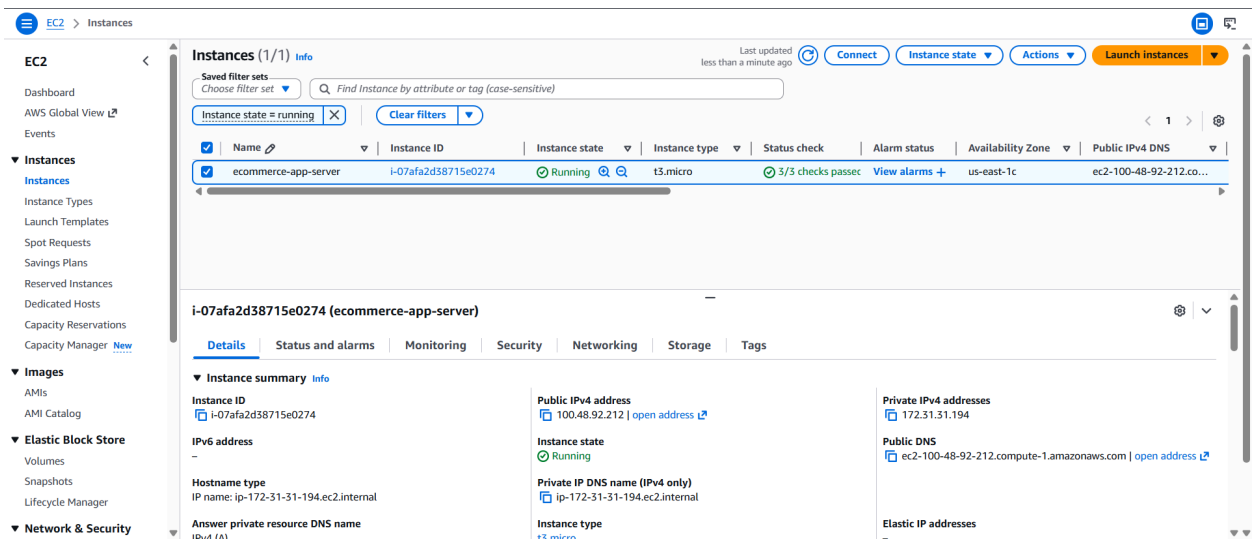
Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

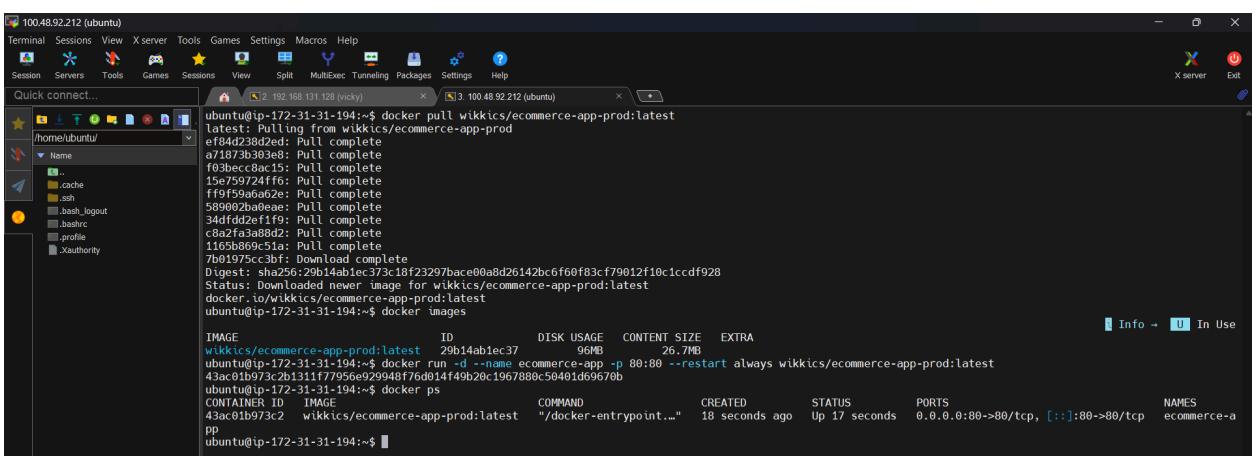
Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without

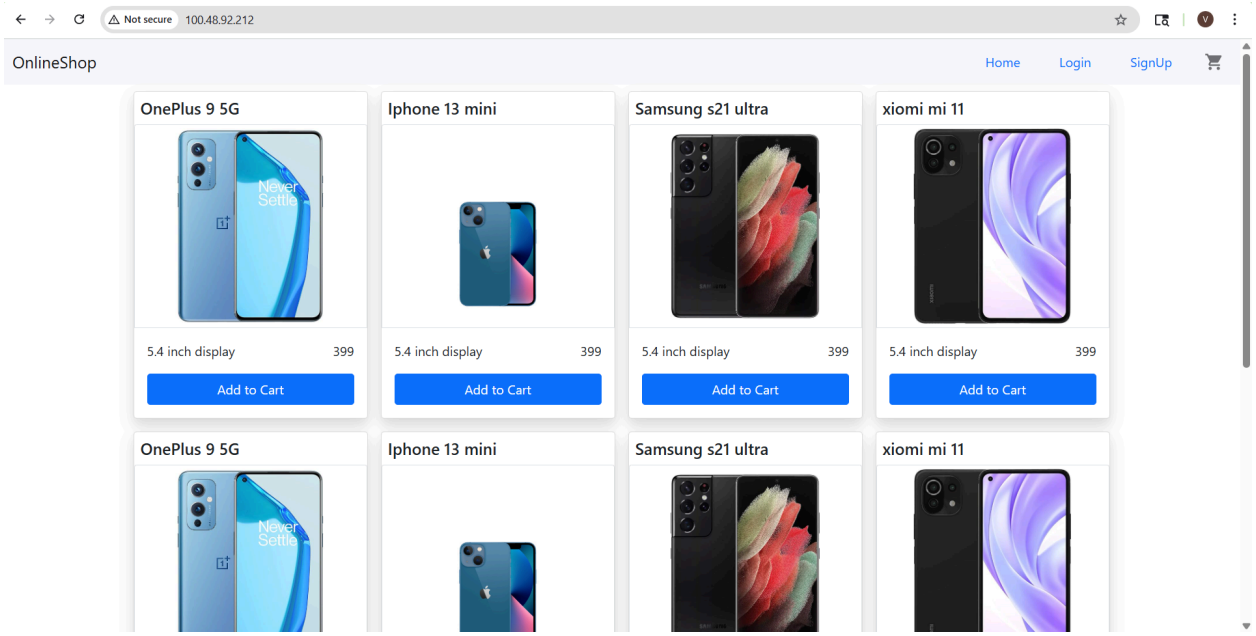
I launched an AWS EC2 t2.micro instance named `ecommerce-app-server` using Ubuntu Server 22.04 LTS. I configured the Security Group with two inbound rules — port 22 (SSH) accessible only from my IP for security and port 80 (HTTP) accessible from anywhere so users can access the application. A key pair named `ecommerce-app-key` was created and downloaded as a `.pem` file for SSH access.

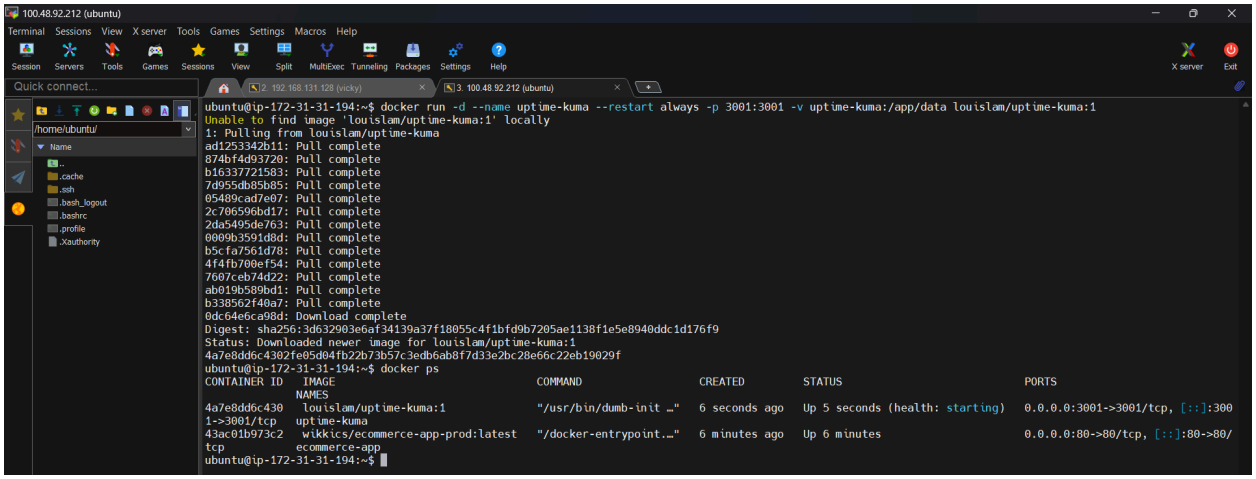
I connected to the EC2 instance from my local machine using SSH with the downloaded key file. After connecting, I installed Docker on the EC2 instance using the official Docker repository. Since the production Docker image was stored in a private Docker Hub repository, I logged into Docker Hub using `docker login` before pulling the image.

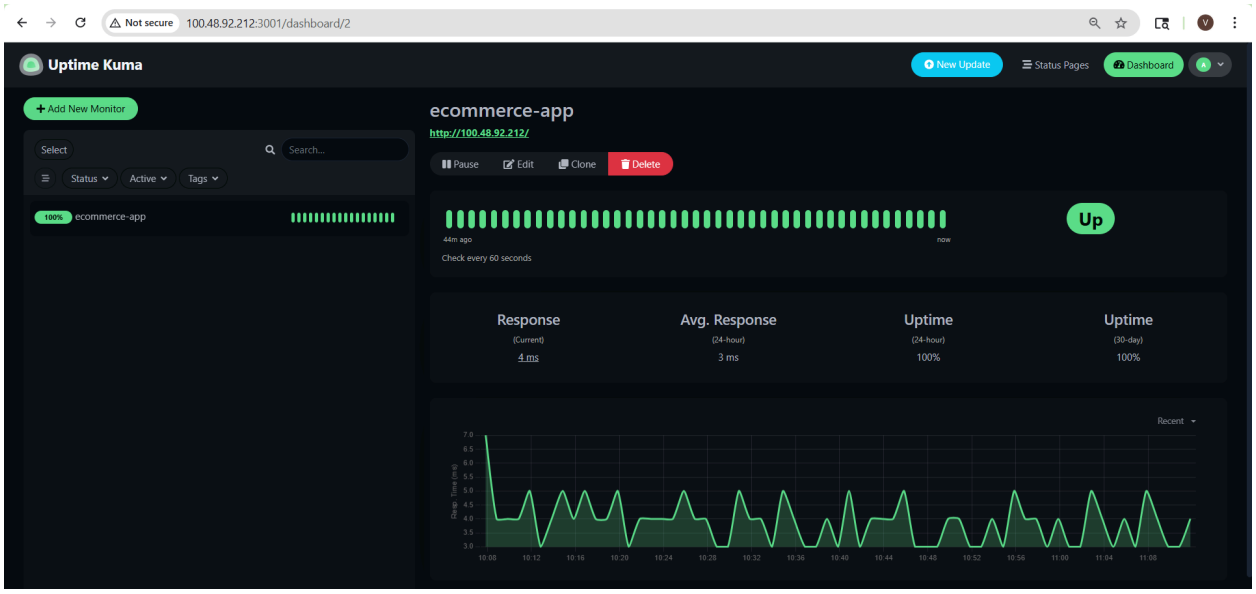
I pulled the production image `wikkics/ecommerce-app-prod:latest` from Docker Hub and ran it as a container on port 80 with `restart always` to ensure it stays running. The ecommerce application was successfully accessible at the EC2 public IP address in the browser.






Not secure
100.48.92.212
OnlineShop
Home
Login
SignUp
OnePlus 9 5G
Iphone 13 mini
Samsung s21 ultra
xiomi mi 11
5.4 inch display
399
Add to Cart


100.48.92.212 (ubuntu)
ubuntu@ip-172-31-31-194:~$ docker run -d --name uptime-kuma --restart always -p 3001:3001 -v uptime-kuma:/app/data louislam/uptime-kuma:1
Unable to find image 'louislam/uptime-kuma:1' locally
1: Pulling from louislam/uptime-kuma
ad1253342b11: Pull complete
874bf4d93720: Pull complete
b16337721583: Pull complete
7d955db85b85: Pull complete
05489cad7e07: Pull complete
2c706596bd17: Pull complete
2da5495de763: Pull complete
0009b3591d8d: Pull complete
b5cfa7561d78: Pull complete
4f4fb700ef54: Pull complete
7607ceb74d22: Pull complete
ab019b589bd1: Pull complete
b338562f40a7: Pull complete
0dc64e6ca98d: Download complete
Digest: sha256:3d632903e6af34139a37f18055c4f1bfd9b7205ae1138f1e5e8940ddc1d176f9
Status: Downloaded newer image for louislam/uptime-kuma:1
4a7e8dd6c4302fe05d04fb22b73b57c3edb6ab8f7d33e2bc28e66c22eb19029f
ubuntu@ip-172-31-31-194:~$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
4a7e8dd6c430 louislam/uptime-kuma:1 "/usr/bin/dumb-init …" 6 seconds ago Up 5 seconds (health: starting) 0.0.0.0:3001->3001/tcp, [::]:3001->3001/tcp uptime-kuma
43ac01b973c2 wikkics/ecommerce-app-prod:latest "/docker-entrypoint.…" 6 minutes ago Up 6 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp ecommerce-app
ubuntu@ip-172-31-31-194:~$

I set up Uptime Kuma as an open-source monitoring tool to check the health status of the ecommerce application. Uptime Kuma was deployed as a Docker container on the same EC2 instance on port 3001. I also added port 3001 to the EC2 Security Group inbound rules to allow access to the monitoring dashboard.

After accessing the Uptime Kuma dashboard at `http://100.48.92.212/:3001`, I created an admin account and added a new HTTP monitor for the ecommerce application with the EC2 public IP as the target URL and a heartbeat interval of 60 seconds. The monitor showed the application status as **UP** with response time displayed on the dashboard.